UNIVERSITÀ DEGLI STUDI DI MILANO

Progettazione di software sicuro

– Eserciziario –

Modulo 1

M1 – UD1 – PROPRIETÀ DEL SOFTWARE SICURO

M1 – UD1 – L1, L2

1.1 Dare la definizione di *processo* e descrivere le *proprietà* relative al processo software.

1.2 Descrivere l'*analisi di sicurezza* nel ciclo di vita del SW.

1.3 Descrivere e classificare la *manutenibilità* del SW.

1.4 Descrivere la differenza tra *processo* e *prodotto* per SW sicuro e descrivere almeno tre *qualità* per il SW sicuro.

1.5 Descrivere la differenza tra *auditing* e *monitoraggio* per SW sicuro.

1.6 Descrivere le qualità esterne di un prodotto SW sicuro.

1.7 Descrivere le tre modalità di modifica di un sistema sicuro nella fase di manutenzione.

1.8 Dare la definizione di *affidabilità* e di *robustezza* per il software.

1.9 Dare la definizione di *prodotto* e *processo* software. Descrivere brevemente le *proprietà* relative a prodotto e processo software.

1.10 Nello sviluppo di SW sicuro, motivare il perché la *sicurezza* è definita come proprietà context-sensitive.

M1 – UD2 – CICLO DI VITA DEL SOFTWARE

M1 – UD2 – L1, L2, L3

1.11 Presentare le diverse fasi del ciclo di vita del SW.

1.12 Descrivere i modelli di ciclo di vita del SW per i processi incrementali.

1.13 Descrivere similitudini e differenze, vantaggi e svantaggi del modello a cascata e del modello a spirale per la progettazione di sistemi sicuri.

1.14 Descrivere il *processo di valutazione* per ottenere la certificazione di SW sicuro in base ai common criteria.

1.15 Descrivere i *modelli evolutivi* per il ciclo di vita del software.

1.16 Descrivere la classificazione dei *requisiti* in fase di specifica.

1.17 Dare la definizione di *artifatti* e *stakeholders* in un processo di sviluppo software.

1.18 Descrivere i modelli di ciclo di vita adatti per lo sviluppo di SW sicuro.

1.19 Descrivere (a) il modello a spirale e (b) il motivo per cui tale modello è ritenuto idoneo per lo sviluppo di SW sicuro.

1.20 Definire le *fasi* in cui è organizzato un processo per lo sviluppo del SW.

1.21 Descrivere la classificazione dei requisiti nell'ambito del processo di sviluppo del software.

1.22 Descrivere *un* modello evolutivo di ciclo di vita del software ed indicarne i vantaggi.

1.23 Descrivere i modelli incrementali per lo sviluppo del software.

M1 – UD3 – VULNERABILITÀ DEL SOFTWARE

M1 – UD3 – L1, L2, L3

1.24 Descrivere il ciclo di vulnerabilità del software.

1.25 Descrivere almeno tre possibili attacchi che possono essere introdotti in fase di progettazione.

1.26 Descrivere gli attacchi possibili a livello di implementazione.

Modulo 2

M2 – UD1 – PRINCIPI DI ARCHITETTURE SICURE

M2 – UD1 – L1, L2, L3

2.1 Presentare il principio guida di *graceful degradation* per architetture sicure.

2.2 Quali sono i principi guida di un'architettura sicura che riguardano malfunzionamenti ed errori del sistema? Descrivi almeno *tre* di questi principi.

2.3 Descrivere il *principio di mediazione completa* per architetture SW.

2.4 Descrivere *tre* dei principi fondamentali per lo sviluppo di architetture sicure.

2.5 Descrivere il *principio di fault safely* delle architetture sicure.

2.6 Descrivere tre *principi* importanti per architetture sicure.

2.7 Descrivere il principio di *auditability* per architetture sicure.

2.8 Dire qual è il criterio per identificare le *funzionalità critiche* per un'architettura sicura.

M2 – UD2 – SCELTE DI TECNOLOGIE SICURE E JAVA

M2 – UD2 – L1, L2, L3

2.9 Descrivere la politica sandbox di un applet in Java 1.4.

2.10 Descrivere come viene gestita la politica di sicurezza in Java2.

2.11 Descrivere il funzionamento della classe *ProtectionDomain* nel meccanismo di sicurezza per Java2.

2.12 Mettere a confronto le funzionalità della classe *SecurityManager* in Java1 ed in Java2.

Modulo 3

M3 – UD2 – MACCHINE A STATI FINITI

M3 – UD2 – L1 – MACCHINE A STATI FINITI DI BASE

3.1 Modellare con una FSM il comportamento di una porta con la chiave inserita. I possibili eventi sono:

- aprire la porta (a);
- chiudere la porta (c);
- girare la chiave per aprire (ga);
- girare la chiave per chiudere (gc).

3.2 Modellare con una FSM il comportamento di una porta girevole in modalità singolo bottone. Se la porta è chiusa, premendo il bottone la porta inizia ad aprirsi. Se si preme ancora una volta il bottone prima che la porta si apre del tutto, la porta inizia a chiudersi. Se, invece, permettiamo alla porta di aprirsi del tutto, una volta aperta la porta inizia a chiudersi automaticamente dopo un timeout di 2-secondi. Per impedire la chiusura automatica e mantenere la porta aperta occorre premere il bottone; a questo punto, un'ulteriore pressione del bottone, fa sì che la porta inizi a chiudersi. Se si preme ancora una volta il bottone prima che la porta si chiuda del tutto, la porta inizia ad aprirsi.

3.3 Modellare con una FSM a stati finiti il comportamento di un CD-player. Il CD-Player ha 3 tasti: OPEN/CLOSE, PLAY e STOP. L'utente può mettere o rimuovere un CD sul piatto. Quando l'utente preme PLAY e c'è un CD sul piatto, il CD-player suona. Quando l'utente preme OPEN/CLOSE, il CD-player si apre o chiude. Quando l'utente preme STOP, il CD-player smette di suonare.

M3 – UD2 – L2 – MACCHINE A STATI FINITI CON OUTPUT

3.4 Modellare con una FSM un automa che fornisce monete da 50 centesimi in cambio di monete da 10 e 20 centesimi. La macchina non fornisce resto.

3.5 Modellare con una FSM il comportamento di un orologio digitale. Un interruttore ON/OFF permette di accendere e spegnere l'orologio. Il tasto set permette di aggiustare l'ora mediante i tasti m ed h per minuti ed ore. Durante la fase di aggiustamento il display segnala l'ora in modo lampeggiante. Settato l'orario, tramite set, l'orologio passa in modalità normale.

M3 – UD2 – L3 – MACCHINE A STATI FINITI ESTESE

3.6 Modellare con una macchina a stati finiti il ciclo di vita di un conto corrente in banca. Gli eventi interessanti sono:

- *deposit(m)* per depositare un certo ammontare m di denaro;
- *withdraw(m)* per prelevare un certo ammontare m di denaro.

E' sempre possibile effettuare prelievo di denaro, fintanto che il bilancio del conto non scende al di sotto di un certo limite *overdraft_limit*. Se si raggiunge tale limite, o se il bilancio si mantiene negativo per più di 3 mesi, il conto corrente viene "bloccato".

3.7 Modellare tramite una FSM estesa il modello di funzionamento di una struttura dati "queue". All'inizio la coda è vuota. Gli eventi interessanti sono:

- *insert_rear* per inserire un elemento in coda;
- *delete_front* per cancellare un elemento dalla testa della coda.

Una variabile globale *size* rappresenta l'effettiva lunghezza della coda. Essa vale 0 quando la coda è vuota, e k (una costante globale) quando è piena.

3.8 Utilizzando una macchina a stati finiti estesa modellate il comportamento del seguente distributore di bevande. Un distributore automatico di bevande consente l'acquisto di una bevanda a scelta mediante apposita chiavetta pre-pagata. Il dispositivo permette all'utente di selezionare la bevanda desiderata tramite la digitazione del codice della bevanda su una tastiera di cui il distributore è fornito. Se la bevanda non è disponibile, tramite display viene visualizzato un opportuno messaggio che la bevanda non è erogabile. Se la bevanda è disponibile, sul display viene visualizzato il costo della bevanda. Se l'utente inserisce la chiavetta pre-pagata entro il tempo massimo di 1 minuto, il costo della bevanda viene sottratto all'ammontare di soldi contenuti nella chiavetta e la bevanda viene erogata. Se la carica della chiavetta non riesce a coprire il costo della bevanda, viene visualizzato un

messaggio che l'erogazione non è possibile per ammontare insufficiente. Se l'utente non inserisce la chiavetta entro il tempo massimo stabilito, la scelta viene annullata ed il distributore ritorna nello stato iniziale di pronto.

M3 – UD2 – L4 – COMMUNICATING MACHINES

3.9 Modellare con il formalismo delle Communicating FSM estese il comportamento di un sistema producer/consumer con buffer di memoria. Il buffer è anch'esso visto come processo che interagisce con le macchine producer e consumer attraverso due canali rispettivamente *deposit* e *remove*. L'attività interna del buffer è di tipo computazionale. Esso contiene una variabile vettoriale *buf* di capacità limitata da 0 a n-1 (con n una costante) usata come buffer circolare per contenere i dati. La variabile *full* mantiene l'effettiva lunghezza del buffer. Due variabili *in* e *out* fanno da indici alla variabile buf: buf[in] è l'elemento ricevuto in input dal producer, mentre buf[out] è l'elemento prodotto in output per essere consumato dal consumer. Inizialmente in=out=full=0.

3.10 Modellare con una macchina di comunicazione temporale il comportamento del seguente sistema di controllo di due sbarre. Il sistema di controllo è costituito da due sbarre collegate ad un computer che, per ciascuna sbarra, ne determina la posizione di aperta (UP) o chiusa (DOWN). Il computer controlla la sequenza di apertura/chiusura delle sbarre emettendo i segnali *open* (= vai a UP) e *closed* (= vai a DOWN) a cui le sbarre rispondono cambiando posizione. Il funzionamento delle sbarre è controllato dal computer in base al seguente ciclo fisso di quattro fasi: fase1, per 50 secondi, entrambe le sbarre sono in DOWN; fase2, per 100 secondi una sbarra è in UP e l'altra è in DOWN; fase3, per 50 secondi entrambe le spie sono in DOWN nuovamente; fase4, per 100 secondi la sbarra che prima era in UP passa a DOWN mentre l'altra passa a DOWN ad UP. Ed il ciclo si ripete.

3.11 Utilizzando le macchine di comunicazione estese, modellare il seguente sistema di controllo di un impianto termico. Il sistema è composto da un controllore, da una caldaia e da un sensore. Il sensore è utilizzato per alternare il funzionamento del sistema termico tra giorno e notte. In fase *giorno*, il controllore invia il comando di accensione alla caldaia se la temperatura ambiente è meno del valore standard settato dall'utente. L'utente può settare a piacere la temperatura dell'ambiente. Per default la temperatura ambiente è fissata a 20°. La caldaia si spegne automaticamente quando arriva in temperatura. In fase *notte*, il controllore

invia alla caldaia un comando di accensione se la temperatura è inferiore ad un valore minimo. Tale valore può essere stabilito a piacere dall'utente. Per default, esso è fissato a 17°.

3.12 Con le macchine a stati finiti di comunicazione modellare la seguente versione semplificata dell'*alternating bit protocol* (ABP). I messaggi sono inviati da una transmitter A ad un receiver B. Inizialmente si assuma che il canale da A a B non contenga messaggi in transito. Ciascun messaggio da A a B ha la forma <*msg*|*bit*> con *msg* sequenza di dati e *bit* un bit (0 o 1) di controllo. B ha due caratteri di conferma (acknowledge characters) che può inviare ad A: ACK0 ed ACK1. Transitando in rete, i messaggi di A a B potrebbero essere corrotti, ma assumiamo che A e B siano in grado di sapere se i messaggi che ricevono sono corrotti o no. Quando A invia un messaggio, lo invia continuamente finché non riceve un acknowledge character da B che contiene lo stesso bit di controllo del messaggio inviato. Quando ciò accade, A inizia a trasmettere un nuovo messaggio. Quando B riceve un messaggio non corrotto ed avente bit di controllo 0, inizia ad inviare ad A l'acknowledge character ACK0 e così procede finché non riceve da A un messaggio valido con bit di controllo 1. In tal caso inizia ad inviare ACK1 ad A.

3.13 Una linea tranviaria unidirezionale incrocia una strada. Due sensori ed un computer controllano la sbarra. Un sensore rileva l'arrivo del treno, mentre l'altro rileva l'avvenuto passaggio. Modellare con il formalismo delle Communicating Machine estese e temporali 5 macchine: una per il treno (Train), 2 per i relativi sensori (Entering-Sensor, Leaving-Sensor), una per la sbarra (Gate), ed una per il sistema di controllo (Controller) vero e proprio. Quest'ultimo, su segnalazione del sensore Entering-Sensor, impiega da 2 a 3 tick di tempo per decidere se abbassare o no la sbarra, e se decide di abbassarla invia un apposito segnale alla sbarra e poi aspetta un segnale dal sensore Leaving-Sensor. La sbarra impiega da 2 a 4 tick per raggiungere la posizione down. Dopo che il treno è passato, il treno invia un segnale al Leaving-Sensor che a sua volta invia un segnale al controllore. Ricevuto quest'ultimo segnale, il sistema di controllo impiega da 1 a 2 tick per decidere ed inviare il segnale alla sbarra per rialzarla. Infine, la sbarra prende da 3 a 4 tick per rialzarsi definitivamente.

3.14 L'apertura e chiusura delle porte di una metropolitana è controllata da un sistema costituito da due sensori (il sensore *arrived* che rileva l'arresto della metro alla stazione, ed il sensore *pass_person* che rileva il passaggio di una persona che vuole entrare nella metro quando le

porte si stanno chiudendo) ed un controller. Su segnalazione del sensore arrived, il controller impiega da 1 a 3 tick di tempo per aprire le porte della metro inviando un apposito segnale alle porte e poi aspetta un segnale dalle porte di avvenuta apertura. La singola porta impiega da 2 a 4 tick per raggiungere la posizione open. Passato un minuto dall'apertura completa delle porte, il controller aziona la chiusura delle porte ed attende il segnale di avvenuta chiusura. La porta prende da 2 a 4 tick per chiudersi definitivamente. Se il sensore pass_person invia un segnale alla porta, la porta reagisce aprendosi completamente. In questo caso, il controller provvederà nuovamente alla chiusura delle porte. Modellare con il formalismo delle Communicating Machine estese e temporali il comportamento del sistema di controllo delle porte di una metropolitana.

3.15 Utilizzando le macchine di comunicazione, modellare il comportamento di un sistema di telesoccorso. Su segnalazione di un utente, la centrale invia all'ambulanza il comando di recarsi ad un dato indirizzo. L'ambulanza impiega da 15 a 30 minuti per raggiungere il posto richiesto. E successivamente impiega da 10 a 30 minuti per portare l'utente in ospedale. Appena l'ambulanza rientra in sede (diversa da ospedale), la centrale può evadere una seconda richiesta, e così via.

3.16 Utilizzando le macchine di comunicazione estese, modellare un sistema composto da due processi che richiedono l'accesso in mutua-esclusione ad una regione critica. L'accesso alla regione è coordinato dall'utilizzo di un semaforo: solo un processo alla volta può trovarsi nella regione critica. Un processo può trovarsi in uno dei seguenti quattro stati:

- *idle*: non vuole entrare nella regione critica;
- *entering*: ha fatto richiesta di entrare nella regione critica e aspetta che il semaforo sia libero;
- *critical*: è nella regione critica con il semaforo occupato (lo occupa il processo quando entra);
- *exiting*: esce dalla regione critica e libera il semaforo.

M3 – UD2 – L5, L6, L7 – MACCHINE DI STATO UML

3.17 Modellare in UML una macchina di stato relativa al ciclo di vita di un applet Java. Un applet è un'applicazione Java progettata per essere eseguita sul Web. Quando un browser web visualizza una pagina (tipo html) che contiene un applet, insieme agli altri elementi della

pagina viene caricato (istanziato) l'oggetto applet. I metodi dell'applet attraverso cui il browser gestisce un oggetto applet sono i seguenti:

- *init()*: chiamato quando l'applet viene caricato nella finestra del browser;
- *start()*: chiamato per avviare l'esecuzione dell'applet all'inizio e ogni qual volta l'applet da uno stato in cui ne viene arrestata l'esecuzione (stopped) passa ad uno stato attivo (running);
- *stop()*: chiamato per arrestare l'esecuzione dell'applet, ad esempio, quando l'utente lascia momentaneamente la pagina web dove l'applet sta girando;
- *destroy()*: chiamato per la rimozione dell'applet da parte del browser quando, per esempio, l'applicazione browser stessa viene chiusa.

3.18 Modellare tramite una macchina di stato UML il funzionamento di un orologio digitale che segnala l'ora come hh:mm. Un interruttore ON/OFF permette di accendere e spegnere l'orologio. Quando viene acceso, il display dell'orologio segnala un orario non attendibile come 00:00. Il tasto set permette di aggiustare l'ora mediante i tasti m ed h per minuti ed ore; durante la fase di aggiustamento, il display segnala l'ora in modo lampeggiante. Settata l'ora, tramite set, l'orologio passa in modalità normale in cui l'orario viene visualizzato in modo non lampeggiante.

3.19 Modellare con una macchina di stato UML il comportamento di una radiosveglia che è composta dalla componente radio e dalla componente sveglia digitale. La radiosveglia può essere accesa o spenta tramite un interruttore ON/OFF. Se accesa, la sveglia segnala l'ora, mentre la radio può essere accesa o spenta. Quando la radiosveglia è accesa, è possibile settare l'ora secondo la seguente modalità. Un tasto set permette di aggiustare l'ora mediante i tasti m ed h per minuti ed ore; durante la fase di aggiustamento, il display segnala l'ora in modo lampeggiante. Settata l'ora, tramite set, l'orologio passa in modalità normale in cui l'orario viene visualizzato in modo non lampeggiante. La radio viene attivata tramite un tasto radio. Quando accesa, la sintonia è per default settata a FM 89, ma è possibile attivare la ricerca automatica per cambiare sintonia tramite un tasto sint.

3.20 Un sistema ISPDialer può connettersi ad un internet service provider (ISP). Il sistema può quindi essere connesso, non connesso o trovarsi nella fase in cui cerca la connessione. Durante il tentativo di connessione, il sistema mette inizialmente il modem in modalità off-hook ed aspetta il segnale di linea libera (dialtone). Se il segnale dialtone non arriva entro

20 secondi, il sistema segnala la mancanza di linea libera e passa in modo non connesso rimettendo il modem in modalità hook. Il sistema viene quindi fermato. Se il segnale di linea libera viene ricevuto entro 20 secondi, il sistema esegue l'attività dialISP in cui cerca di connettersi all'ISP. Appena tale attività termina, il sistema aspetta il carrier da parte dell'ISP. Se questo segnale non arriva entro 20 secondi, il sistema segnala la mancanza di carrier da parte dell'ISP e va in modo non connesso rimettendo il modem in modalità hook. Se il segnale arriva entro 20 secondi, il sistema viene connesso ed utilizza la connessione finché desidera. Al termine della connessione il sistema viene fermato rimettendo il modem in modalità hook. Se in qualsiasi istante in cui cerca la connessione, il sistema riceve il segnale cancel, il sistema passa in stato non connesso. Disegnare la macchina di stato UML che formalizza il comportamento del sistema ISPDialer.

3.21 Una piccola società S.r.l. di formazione offre un seminario a settimana. Ogni settimana del calendario di insegnamento è un'istanza della classe *SettimanaSeminario*. Lo stato di prenotazione di una data settimana può essere *disponibile*, *prenotataProvvisoria* e *prenotataDefinitiva*. Quando viene creata un'istanza di SettimanaSeminario (ossia viene collocata sul calendario), lo stato iniziale è disponibile. Un cliente può prenotare una settimana con o senza il versamento di un anticipo (tramite l'operazione clientePrenotaSettimana(cliente, anticipo)). In caso di anticipo, il cliente ha diritto ad una prenotazione confermata (prenotataDefinitiva) altrimenti può ottenere solo una prenotazione provvisoria. Disegnare la porzione di diagramma degli stati UML per i tre stati di SettimanaSeminario appena descritti.

3.22 Utilizzando il meccanismo UML di sincronizzazione *eventi-azioni*, modellare il comportamento di un *processo* che richiede l'accesso ad una risorsa condivisa. L'accesso alla risorsa è coordinato da un *semaforo* che concede l'accesso alla risorsa solo se disponibile (stato del semaforo *verde*). Quando la risorsa è bloccata dall'uso del processo il semaforo è *rosso*. Il processo può trovarsi in uno dei seguenti quattro stati:

- *idle*: stato di disponibilità;
- *entering*: ha fatto richiesta della risorsa condivisa e aspetta che il semaforo sia verde;
- *action*: ha in uso la risorsa con il semaforo occupato (lo occupa il processo quando entra in action).

Quando è in action, il processo svolge tre attività parallele al termine delle quali rilascia la risorsa, libera il semaforo e torna in idle. Il verificarsi di un errore del sistema, mette il

processo in stato di *errore*. Il ripristino dallo stato di errore, riporta il processo allo stato dell'interruzione.

3.23 Utilizzando le macchine di stato UML per sistemi a componenti, modellare un sistema composto da due processi che richiedono l'accesso in mutua-esclusione ad una regione critica. L'accesso alla regione è coordinato dall'utilizzo di un semaforo: solo un processo alla volta può trovarsi nella regione critica. Un processo può trovarsi in uno dei seguenti quattro stati:

- *idle*: non vuole entrare nella regione critica;
- *entering*: ha fatto richiesta di entrare nella regione critica e aspetta che il semaforo sia libero;
- *critical*: è nella regione critica con il semaforo occupato (lo occupa il processo quando entra);
- *exiting*: esce dalla regione critica e libera il semaforo.

3.24 Utilizzando le macchine di stato UML modellare il comportamento di un *montacarichi* che si muove su tre piani – top, middle, low – in base alla richiesta che arriva da sensore. Per esempio, se il sensore segnala top, il montacarichi raggiunge la posizione più alta. L'elevatore è dotato di un tasto per aprire le porte se il montacarichi è fermo, ed un tasto di emergenza che se pigiato causa il suono di una sirena e mette il montacarichi "fuori servizio". Quando è fuori servizio, se ripristinato, il montacarichi riparte dall'ultima configurazione lasciata.

3.25 Utilizzando il meccanismo di sincronizzazione eventi-azioni delle macchine di stato UML, modellare il comportamento di un elevatore comandato da un controller. Il controller comanda l'elevatore in base alla richiesta di avere l'elevatore ai piani top, middle, e low che avviene attraverso opportuni tasti (esterni) posti ai tre livelli. L'elevatore è dotato di un tasto per aprire le porte quando è fermo, e un tasto di emergenza che se pigiato causa il suono di un allarme e mette l'elevatore "fuori servizio". Quando l'elevatore è fuori servizio, due processi paralleli vengono attivati per il controllo dci dispositivi interni ed esterni all'ascensore. Terminata la fase di revisione, l'elevatore torna alla configurazione che aveva prima dell'interruzione.

3.26 Modellare con una macchina di stato UML il seguente comportamento di una macchina per l'imbottigliamento. La macchina, una volta accesa, richiede una bottiglia ed aspetta che la bottiglia sia stata posizionata nella sede apposita, dove un sensore rileva la sua presenza. A questo punto, inizia il riempimento della bottiglia. Quando il liquido raggiunge un livello stabilito, un sensore addizionale viene attivato, la bottiglia viene tappata, ed il sistema richiede che la bottiglia venga rimossa. Quando al sistema arriva il segnale che la bottiglia è stata rimossa, una nuova bottiglia viene richiesta, ed il ciclo ricomincia. E' possibile interrompere manualmente il processo di imbottigliamento. In questo caso, la bottiglia viene tappata e viene generato un segnale di allarme. Il sistema può essere spento solo quando è in attesa di una nuova bottiglia.

3.27 Utilizzando il formalismo delle macchine di stato UML, modellare il pagamento di pedaggio autostradale tramite telepass. All'arrivo di un'auto al casello autostradale, comunicata tramite sensore, un computer invia al sistema telepass l'informazione della targa della macchina e l'importo da pagare. Il sistema, fatti i dovuti controlli, invia al computer accettazione o rifiuto della richiesta. Ricevuta la risposta, il computer invia un segnale sul display di rifiuto del pagamento, oppure il comando di apertura alla sbarra. Questa provvede ad aprirsi ed a chiudersi dopo un minuto a seguito di opportuno comando del computer.

3.28 Utilizzando le macchine di stato UML modellare il seguente sistema di illuminazione automatica di una stanza. Il sistema è composto da una plafoniera di luci e da due sensori *sensore-luce* e *sensore-movimento*. Il sensore-luce è utilizzato per disattivare il funzionamento del sistema di illuminazione durante il giorno. In fase *notte*, si attiva la luce di emergenza nella stanza. Se il sensore-movimento rileva la presenza di una persona nella stanza, le luci della stanza vengono completamente accese. La luce nella stanza rimane accesa fino a 5 minuti dopo che la persona si è allontanata dalla stanza. La luce ritorna in modalità emergenza se in fase notte.

3.29 Sfruttando il meccanismo di sincronizzazione tramite eventi, modellare con una macchina di stato UML il comportamento della sbarra (Gate) e del sistema di controllo (Controller) di una linea tranviaria che incrocia una strada. Il sistema di controllo, su segnalazione del sensore Entering che avvisa dell'arrivo di un treno, dopo 50 secondi invia un apposito segnale alla sbarra di chiudersi e poi aspetta un segnale dal sensore Leaving che avvisa quando il treno è passato. La sbarra impiega 30 secondi per raggiungere la posizione down.

Dopo che il treno è passato, il sensore Leaving invia un segnale al controllore. Ricevuto quest'ultimo segnale, dopo 50 secondi il sistema di controllo invia il segnale alla sbarra per rialzarla. La sbarra impiega 30 secondi per rialzarsi definitivamente.

3.30 Sfruttando il meccanismo di sincronizzazione tramite eventi, modellare con una macchina di stato UML il comportamento di un sistema d'irrigazione automatica. Il sistema è composto da un controllore e da un annaffiatoio. In modalità inverno il controllore invia il comando di erogazione dell'acqua ogni 12 ore e l'annaffiatoio eroga acqua per 15 minuti, dopo di che riceve il comando di interruzione di erogazione acqua. In modalità inverno l'erogazione dell'acqua è richiesta ogni 8 ore per la durata di 30 minuti. Il controllore è dotato di un sensore di umidità. Se il segnale che arriva da tale sensore è alto, l'erogazione dell'acqua, per il ciclo corrente, è sospesa; se il segnale è basso, l'acqua è erogata, nel ciclo corrente, per ulteriori 5 minuti.

3.31 Modellare con una macchina di stato UML il seguente comportamento di una macchina per forgiare metalli. La macchina, una volta accesa, richiede un pezzo di metallo grezzo (ossia non forgiato) ed aspetta che questo sia stato posizionato nella sede apposita, dove un sensore rileva la sua presenza. A questo punto, inizia la fase di riscaldamento del pezzo di metallo. Quando la temperatura raggiunge un livello stabilito, inizia la fase di forgiatura. Quando questa termina, un sensore addizionale viene attivato, ed il sistema richiede che il pezzo forgiato venga rimosso. Quando al sistema arriva il segnale che il pezzo forgiato è stato rimosso, un nuovo pezzo grezzo viene richiesto, ed il ciclo ricomincia. È possibile interrompere manualmente il processo di forgiatura. In questo caso, viene generato un segnale di allarme. La macchina può essere spenta solo quando è in attesa di un nuovo pezzo grezzo da forgiare.

M3 – UD3 – DESIGN BY CONTRACT

M3 – UD3 – L1, L2, L3

3.32 Descrivere i principi su cui si basa il design by contract e portare un esempio di contratto su un metodo Java che computa la somma degli elementi di un array.

3.33 (a) Descrivere a cosa servono precondizioni, postcondizioni ed invarianti nel Design by Contract. (b) Scrivere la segnatura e l'opportuno contratto JML per un metodo che data una sequenza di interi, la ripartisca in due sequenze di interi positivi ed interi negativi.

3.34 (a) Descrivere l'uso di precondizioni, postcondizioni ed invarianti per il Design by Contract. (b) Definire una classe Java avente come attributo un array di interi A e come metodo una funzione booleana *pospredomina* che restituisce true se la somma degli interi positivi in A supera il valore assoluto della somma degli interi negativi di A. (c) Definire un contratto JML per il metodo *pospredomina*.

3.35 (a) Descrivere l'uso di precondizioni, postcondizioni ed invarianti per il Design by Contract. (b) Definire una classe Java avente come attributo un array di interi A e come metodo una funzione booleana *sommaPositiva* che restituisce true se la somma degli interi in A è positiva. (c) Definire un contratto JML per il metodo *sommaPositiva*.

3.36 (a) Descrivere l'uso di precondizioni, postcondizioni ed invarianti per il Design by Contract. (b) Definire una classe Java avente come attributi tre array di interi A, B, e C, ed un metodo ripartisci che ripartisce l'array A negli array, ordinati in modo crescente, B di interi positivi e C di interi negativi. (c) Dare la segnatura e l'opportuno contratto JML per il metodo ripartisci.

Modulo 4

M4 – UD1 – SICUREZZA DEI LINGUAGGI DI PROGRAMMAZIONE

M4 – UD1 – L1, L2, L3

4.1 Descrivere almeno 3 caratteristiche del linguaggio C che possono causare problemi di sicurezza e motivare la risposta.

4.2 Descrivere in maniera sintetica il *type checking* del linguaggio Java.

4.3 Presentare vantaggi e svantaggi del linguaggio C rispetto alla sicurezza.

4.4 Quali sono i criteri che inducono alla scelta tra Java e C come linguaggio di codifica per lo sviluppo di un'applicazione sicura?

4.5 Confrontare i linguaggi C e Java in termini di sicurezza.

4.6 Descrivere i problemi di violazione di tempo e di spazio causati dai puntatori del linguaggio C e che possono causare problemi di sicurezza.

M4 – UD2 – DALLA SPECIFICA AL CODICE

M4 – UD2 – L1 – MACCHINE A STATI FINITI IN JAVA

4.7 Descrivere il meccanismo degli state pattern per la codifica di una FSM.

4.8 Descrivere come si implementa in Java una macchina a stati finiti di base utilizzando il metodo degli state pattern. Portare un semplice esempio di applicazione.

4.9 Data la FSM dell'**Esercizio 3.5**, darne la rappresentazione in state pattern.

4.10 Data la seguente FSM
S={s1,s2,s3,s4}; I={a,b}; O={0,1}; T={(s1,a,1,s4),(s2,b,0,s3),(s3,a,1,s4),(s4,a,0,s2)}
darne la rappresentazione grafica e quella in state pattern.

4.11 Data la macchina di stato FSM avente

S={s0,s1,s2}; I={a,b,c}; O={0,1};

T={(s0,a,1,s1),(s1,a,0,s0),(s1,b,0,s2),(s2,c,0,s2),(s2,b,1,s0)}

darne la codifica Java mediante state pattern.

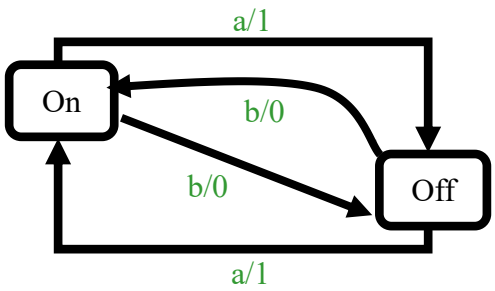
4.12 Data la macchina di stato FSM avente

S={s0,s1,s2}; I={a,b,c}; O={0,1};

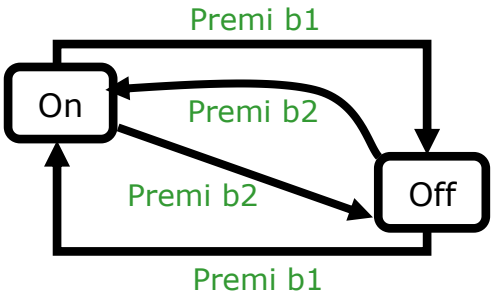
T={(s0,a,1,s1),(s1,a,0,s0),(s1,b,0,s2),(s2,a,0,s2),(s2,c,1,s0)}

darne la rappresentazione mediante state pattern.

4.13 Data la seguente FSM di Mealy, darne la rappresentazione in state pattern.



4.14 Data la seguente FSM, dare la sua rappresentazione in state pattern.



Modulo 5

M5 – UD1 – IL TESTING NEL CICLO DI VITA DEL SW

M5 – UD1 – L1, L2, L3, L4

5.1 Descrivere cosa si intende per *program-based testing* e *specification-based testing*. Definire i criteri per la copertura del program-based testing.

5.2 Definire cosa sono i test di accettazione, conformità e integrazione.

5.3 Descrivere le possibili classificazioni di testing a seconda del *livello* a cui si effettua, del tipo di *accesso* e degli *aspetti* da testare.

5.4 Classificare l'attività di testing rispetto ai requisiti, all'architettura ed al codice.

5.5 Definire la differenza tra *testing funzionale* e *testing di sicurezza* in sistemi software sicuri.

M5 – UD2 – FONDAMENTI TEORICI DEL TESTING

M5 – UD2 – L1, L2

5.6 Dare la definizione di criterio affidabile, valido e ideale. Enunciare il teorema di Goodenough e Gerhart.

5.7 Dare la definizione di criterio valido, affidabile ed ideale. Determinare un criterio ideale per il seguente programma che dovrebbe restituire il valore della funzione `f(x,y)=x*(y*10)` per valori x,y interi, crescenti e maggiori di 1, 0 altrimenti, ma che contiene un errore:

```
int foo (int x, int y) {
   if (x>1 & y>x) return x*(y+10);
     else return 0;
}
```

5.8 Dare la definizione di criterio valido, affidabile ed ideale. Dato il seguente programma che dovrebbe restituire il valore della funzione `foo(x,y)=2(x+y)` per ogni valore x,y interi:

```
int foo (int x, int y) {
   if (x>0 & y>x) return 2*x+y;
      else return 2*(x+y);
}
```

determinare un criterio ideale per il test della funzione.

5.9 Dato il seguente programma che dovrebbe fare la somma di due numeri, ma che contiene un difetto:

```
int somma (int x, int y) {
   if (x>0 && y>x) return x*y;
     else return x+y;
}
```

determinare un criterio per la costruzione di test set ideali.

5.10 Dare la definizione di criterio valido, affidabile ed ideale. Data la seguente funzione raddoppia che restituisce 2x per input positivi ed x altrimenti:

```
int raddoppia (int x) {
if (x>0) return 2+x;
   else return x;
}
```

classificare i seguenti test set in affidabili, validi ed ideali.

T1={{0},{0,2}}; T2={{0,1},{2,3}}; T3={t | t numero dispari}; T4={t | t numero dispari}.

M5 – UD3 – TESTING BASATO SUI PROGRAMMI

M5 – UD3 – L1, L2, L3, L4

5.11 Dare la definizione di *test set adeguato per il criterio di copertura delle istruzioni*. Fare anche un semplice esempio.

5.12 Dato il seguente pseudo-codice:

```
read (a,b,c)
while (a>0 & b>0 & c>0) do
   if (a>2 || (a>b & a>c)) then
         foo1(a,b,c);
     else foo2(a,b,c);
   endif
   a:=a-10;
 endwhile
```

```
end
```

(a) Disegnare il grafo di flusso di controllo.

(b) Stabilire un criterio di copertura per ottenere il branch coverage.

(c) Usare l'MCDC per trovare un test set per la copertura della guardia dell'`if`.

5.13 Determinare il grafo di flusso e definire criteri di copertura per branch e decisioni per il seguente programma:

```
a:=x; b:=y;
while (a<>b) do
   if (a>b) then a:=a-b;
     else b:=b-a;
   endif
endwhile
end
```

5.14 Scrivere i casi di test secondo l'MCDC per la seguente espressione: (a & b) || (c & d).

5.15 Studiare con MCDC la copertura delle condizioni nel seguente statement di if annidati. Determinare la test suite minimale.

```
if (x>5 | y>0 | z>3) then
   if (x>5) then …
      else if (y>0) then …
        else if (z>3) then …
```

5.16 Fare l'analisi delle decisioni e delle condizioni con l'MCDC per il seguente frammento di programma.

```
foo (int x, int y, int z, int w) {
   if ((x>5 && y>0) || (z>5))
   then if (x>5 && z>5) then statement1;
         else statement 2;
   else statement3;
}
```

5.17 Studiare la copertura delle condizioni nel seguente statement di if annidati. Determinare la test suite minimale.

```
if (x>5 & y>0) | (z>5) then
   if (z>5 & v<z) then …
      else if (x>0 & y=4) then …
```

5.18 Quali dei due metodi, tra MCC con short circuit evaluation ed MCDC, determina la test suite minimale per la seguente espressione (x<0 & y=4 & z>17) || x>5. Applicare i due metodi e calcolare la test suite.

5.19 Scrivere i casi di test secondo l'MCDC per l'espressione (x>1 & y>1) || (w=1 & z= 0).

5.20 Calcolare la test suite per testare le condizioni secondo l'MCDC nella seguente espressione: ((x>5 && y>0) || (z>5) || v<z).